

VisionInspect AI

Frontend Development — Backend Integration Prep, Rich Inspection Results & Landing Experience

Milestone 2 (Week 3 & 4) — Individual Contribution Report

Ayusha

Infosys Springboard Virtual Internship Program

Table of Contents

Table of Contents.....	2
1. Project Description	3
2. Backend Integration & Data Contract	3
3. UI/UX Requirement Analysis.....	3
4. Design & Planning.....	3
5. Proposed System (Updated Frontend Architecture)	4
6. Implementation Pipeline.....	4
7. Conclusion	5

1. Project Description

This report covers Milestone 2 (Week 3 & 4) of the frontend contribution to VisionInspect AI, an AI-powered manufacturing defect detection and quality inspection platform developed under the Infosys Springboard Virtual Internship Program (VIP). Milestone 1 delivered the standalone login, image upload, and a basic role-based dashboard shell. Milestone 2 builds directly on that foundation, focused on three areas: preparing the frontend for real backend integration, building a complete inspection results view, and adding a polished landing experience and role-specific analytics based on mentor and team lead feedback.

Backend and detection model development were in progress in parallel during this milestone, so frontend work was structured to be demo-ready on mock data while remaining a near-drop-in fit for the real API once available.

2. Backend Integration & Data Contract

The team lead requested that the upload page be connected to the backend API, or mocked realistically if the backend was not yet ready. Since backend endpoints were still under development at the time of this milestone, the following approach was taken:

- Sent a request to the backend team specifying the exact API contract needed: endpoint and method, request format, and the response fields required — prediction, confidence, severity score/level, pass/fail decision, processed image, and heatmap data.
- Rewrote the mock data layer (`lib/api.js`) to mirror that expected contract exactly, including inline documentation of the real `fetch()` call that will replace the mock logic once the endpoint is live.
- Isolated all mock logic to a single function (`runInspection`), so that integrating the real backend requires no changes to any UI component.

3. UI/UX Requirement Analysis

Two rounds of feedback shaped this milestone's scope:

- Mentor feedback: the Quality Engineer and Factory Supervisor dashboards looked too similar and needed visibly different interfaces reflecting their different responsibilities.
- Team lead feedback: connect the upload flow to the backend (or mock it convincingly), and display the uploaded image, processed image, prediction, severity, confidence, heatmap, and pass/fail result together on the dashboard, with optional polish such as a loading state and image preview.
- A separate mentor suggestion to use a scroll-driven, narrative-style presentation layer (referencing "Narrator"-style scrollytelling and a scroll-based reference project) was evaluated for where it would add value without harming usability of the working dashboards.

4. Design & Planning

Key design decisions for this milestone:

- Scroll-driven storytelling was deliberately scoped to a new marketing/explainer landing page only, not the login or dashboard screens — a landing page benefits from a cinematic, exploratory feel, while working dashboards need to stay fast and information-dense for repeat daily use.

- The landing page narrative was built around the platform's own 5-stage inspection pipeline from the system architecture (Preprocessing, Feature Extraction, Defect Detection, Classification & Severity, Quality Decision), so the content is accurate to the real system rather than generic marketing copy.
- The inspection result view was redesigned around a side-by-side image comparison (uploaded vs. processed-with-heatmap) plus a metrics row, replacing the previous small result cards, to match the level of detail requested by the team lead.
- Additional dashboard analytics (defect breakdown, CSV export, line health, defect distribution, critical-defect alerts) were added on top of the explicit requirements, based on what a Quality Engineer or Factory Supervisor would realistically want from a working tool, not just what was directly requested.

5. Proposed System (Updated Frontend Architecture)

New and updated components introduced this milestone:

- Landing Page (pages/index.js): scroll-narrated walkthrough of the 5 inspection stages, using a reusable ScrollReveal component (IntersectionObserver-driven fade/slide-in) and custom animated SVG visuals per stage (StageVisuals.js).
- Inspection Result Panel (InspectionResult.js): displays the uploaded image and a processed image with a simulated defect heatmap overlay (HeatmapOverlay.js), alongside prediction, confidence, severity, and pass/fail.
- Defect Breakdown (DefectBreakdown.js): a live chart of defect types across a Quality Engineer's logged inspections.
- Toast (Toast.js): a lightweight alert shown when a Critical-severity defect is detected.
- CSV export utility (lib/csv.js): used by both the inspection log and the escalation queue to export records for reporting.
- Factory Supervisor dashboard extended with Production Line Health cards and a Plant-Wide Defect Distribution chart, alongside the existing trend chart and escalation queue.

6. Implementation Pipeline

Backend Integration Prep

- Defined and documented the expected API contract in lib/api.js.
- Requested the confirmed contract from the backend team.
- Expanded mock inspection data to include prediction, confidence, and heatmap fields, in addition to the existing severity scoring.

Inspection Result View

- Built the side-by-side uploaded/processed image comparison.
- Built the CSS-based heatmap overlay simulating a defect focus region.
- Added a confidence progress bar and consolidated result metrics.

Landing Page

- Built the scroll-reveal system and five stage sections with custom animated visuals.
- Added a scroll progress rail and a call-to-action into the login flow.

Dashboard Analytics

- Added defect breakdown chart, log search/filter, and CSV export to the Quality Engineer dashboard.
- Added line health cards, defect distribution chart, and CSV export to the Factory Supervisor dashboard.
- Added a critical-defect toast notification.

Verification

- Verified a clean production build (npm run build) after each major addition, with no errors.
- Manually tested both role dashboards, the upload-to-result flow, CSV exports, and the landing page scroll sequence on the live deployment.

7. Conclusion

Milestone 2 moves the frontend from a functional shell to a demo-ready application: the data layer now mirrors the exact contract requested from the backend team, the inspection result view matches the full set of fields the team lead asked for, and both role dashboards include analytics that go beyond the minimum requirement. The scroll-narrated landing page addresses the mentor's feedback about visual polish, applied specifically where it adds value rather than across the whole application.

The mock data layer remains fully isolated, so the primary remaining task — wiring in the real detection API once the backend team confirms the contract — is expected to be a small, contained change rather than a rebuild of any existing screen.